

Guía: Programación del ESP32 para el TP Seguidor Solar

Este documento describe cómo leer sensores, calcular variables y enviar datos a la página web (`/save\_data.php`) para q

Resumen de la arquitectura

- Sensores: 4x LDR (ADS1115 recomendado), ACS712 (corriente), divisores para Vpanel y Vbat.
- Control: ESP32 (DevKit)
- Comunicaciones: ESP32 -> servidor web (POST JSON a `/save\_data.php`). La página obtiene la última muestra en `/da

Formato JSON esperado

Ejemplo de payload que el ESP32 debe enviar (Content-Type: application/json):

```
...
{
  "timestamp": "2026-05-20T12:00:00Z",
  "LDR_NW": 123,
  "LDR_NE": 120,
  "LDR_SW": 110,
  "LDR_SE": 115,
  "H": 5,
  "servo_angle": 90,
  "Vpanel": 4.8,
  "Vbat": 3.9,
  "I": 0.18,
  "P": 0.864,
  "irradiance": 150.0,
  "soc": 78,
  "charging": true
}
...
```

`index.html` y `script.js` ya obtienen `/data.json` periódicamente y muestran los campos anteriores.

Código ejemplo (Arduino core para ESP32)

Archivo `esp32\_full\_example.ino` (librerías: `Wire`, `Adafruit\_ADS1X15`, `WiFi.h`, `HTTPClient.h`, `WiFiClientSecure.h`):

```
```cpp
// ESP32 - Ejemplo completo (esqueleto)
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <Wire.h>
#include <Adafruit_ADS1X15.h>

const char* ssid = "TU_SSID";
const char* pass = "TU_PASS";
const char* server = "https://optoelectronica.agustingimenez.ar/save_data.php";

Adafruit_ADS1115 ads; // I2C ADS1115

// Calibraciones y pines
const int pinVpanel = 34; // ADC1
const int pinVbat = 35; // ADC1
const int pinACS = 32; // ADC1 (salida del ACS712)

void setup(){
 Serial.begin(115200);
 Wire.begin();
 ads.begin();
}
```

```

WiFi.begin(ssid, pass);
while(WiFi.status() != WL_CONNECTED) delay(200);
}

float readADCVoltage(int pin, float vref=3.3){
 int raw = analogRead(pin);
 return (raw / 4095.0) * vref; // ADC12 bit en ESP32
}

void loop(){
 // Leer LDRs por ADS1115
 int16_t nw = ads.readADC_SingleEnded(0);
 int16_t ne = ads.readADC_SingleEnded(1);
 int16_t sw = ads.readADC_SingleEnded(2);
 int16_t se = ads.readADC_SingleEnded(3);

 // leer tensiones
 float vpanel_adc = readADCVoltage(pinVpanel);
 float vbat_adc = readADCVoltage(pinVbat);

 // aplicar divisores (ajustar valores reales)
 float Vpanel = vpanel_adc * (127.0/27.0); // ejemplo R1=100k,R2=27k
 float Vbat = vbat_adc * 2.0; // R1=100k,R2=100k

 // Leer ACS712 y calcular corriente
 float vacs = readADCVoltage(pinACS);
 float Voffset = 2.5; // medir sin carga
 float sensitivity = 0.066; // V/A para 30A module (ajustar)
 float I = (vacs - Voffset) / sensitivity;

 // Calcular potencia e irradiancia estimada
 float P = Vpanel * I;
 float area = 0.11 * 0.069; // m2 del panel ejemplo
 float eta = 0.15;
 float irradiance = (P > 0.001) ? (P / (area * eta)) : 0.0;

 // H (error horizontal)
 float H = (ne + se) - (nw + sw);

 // preparar JSON
 String payload = "{";
 payload += "\"timestamp\": \"" + String((unsigned long)millis()) + "\", ";
 payload += "\"LDR_NW\": " + String(nw) + ", ";
 payload += "\"LDR_NE\": " + String(ne) + ", ";
 payload += "\"LDR_SW\": " + String(sw) + ", ";
 payload += "\"LDR_SE\": " + String(se) + ", ";
 payload += "\"H\": " + String(H) + ", ";
 payload += "\"servo_angle\": 90, ";
 payload += "\"Vpanel\": " + String(Vpanel) + ", ";
 payload += "\"Vbat\": " + String(Vbat) + ", ";
 payload += "\"I\": " + String(I) + ", ";
 payload += "\"P\": " + String(P) + ", ";
 payload += "\"irradiance\": " + String(irradiance) + ", ";
 payload += "\"soc\": 50, ";
 payload += "\"charging\": false";
 payload += "}";

 // Envío HTTPS
 WiFiClientSecure client;
 client.setInsecure();
 HTTPClient https;

```

```
https.begin(client, server);
https.addHeader("Content-Type","application/json");
int code = https.POST(payload);
String resp = https.getString();
https.end();

delay(1500);
}

```

Notas:

- Ajustar divisores y sensibilidad del ACS712 con mediciones reales.
- Medir Voffset del ACS712 al arrancar (sin corriente) y usarlo.
- Si usás ADS1115, lee las cuentas y conviértelas a voltaje según su ganancia.

## Alternativas de integración

- ESP32 -> POST JSON (API): simple y robusto. El servidor guarda `data.json` y la web la consume.
- ESP32 sirve la página: útil para despliegues off-grid (no requiere servidor externo).
- WebSocket: para actualización en tiempo real (menor latencia). Requiere implementar un servidor WebSocket en el bac

## Seguridad y producción

- En producción, validar certificados en el ESP32 (no usar `setInsecure()`), o usar fingerprint/CA.
- Autenticación: si la página no debe ser pública, añadir token o autenticación básica en la API.

---

Fin de la guía rápida. El archivo `guide.html` contiene una versión navegable.